



US 20250284486A1

(19) **United States**

(12) **Patent Application Publication**
Oberle et al.

(10) **Pub. No.: US 2025/0284486 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **SYSTEM AND METHOD FOR
AUTOMATICALLY ENABLING FEATURES
IN A SOFTWARE PRODUCTION
ENVIRONMENT**

(52) **U.S. Cl.**
CPC **G06F 8/658** (2018.02)

(71) Applicant: **BANK OF AMERICA
CORPORATION**, Charlotte, NC (US)

(72) Inventors: **Michael J. Oberle**, Waxhaw, NC (US);
Rashida Najjee Stephens, Fairburn,
GA (US)

(73) Assignee: **BANK OF AMERICA
CORPORATION**, Charlotte, NC (US)

(21) Appl. No.: **18/597,589**

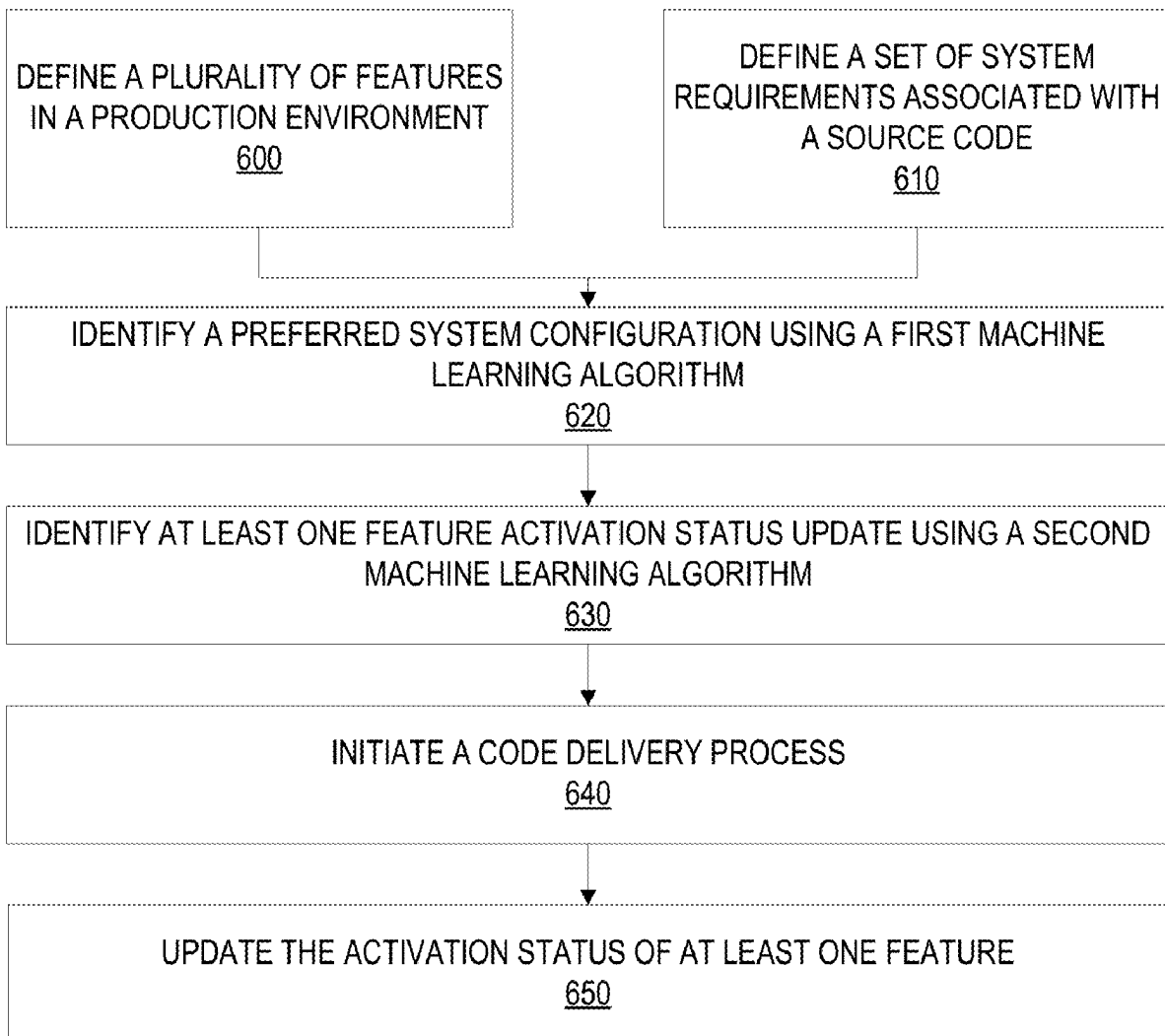
(22) Filed: **Mar. 6, 2024**

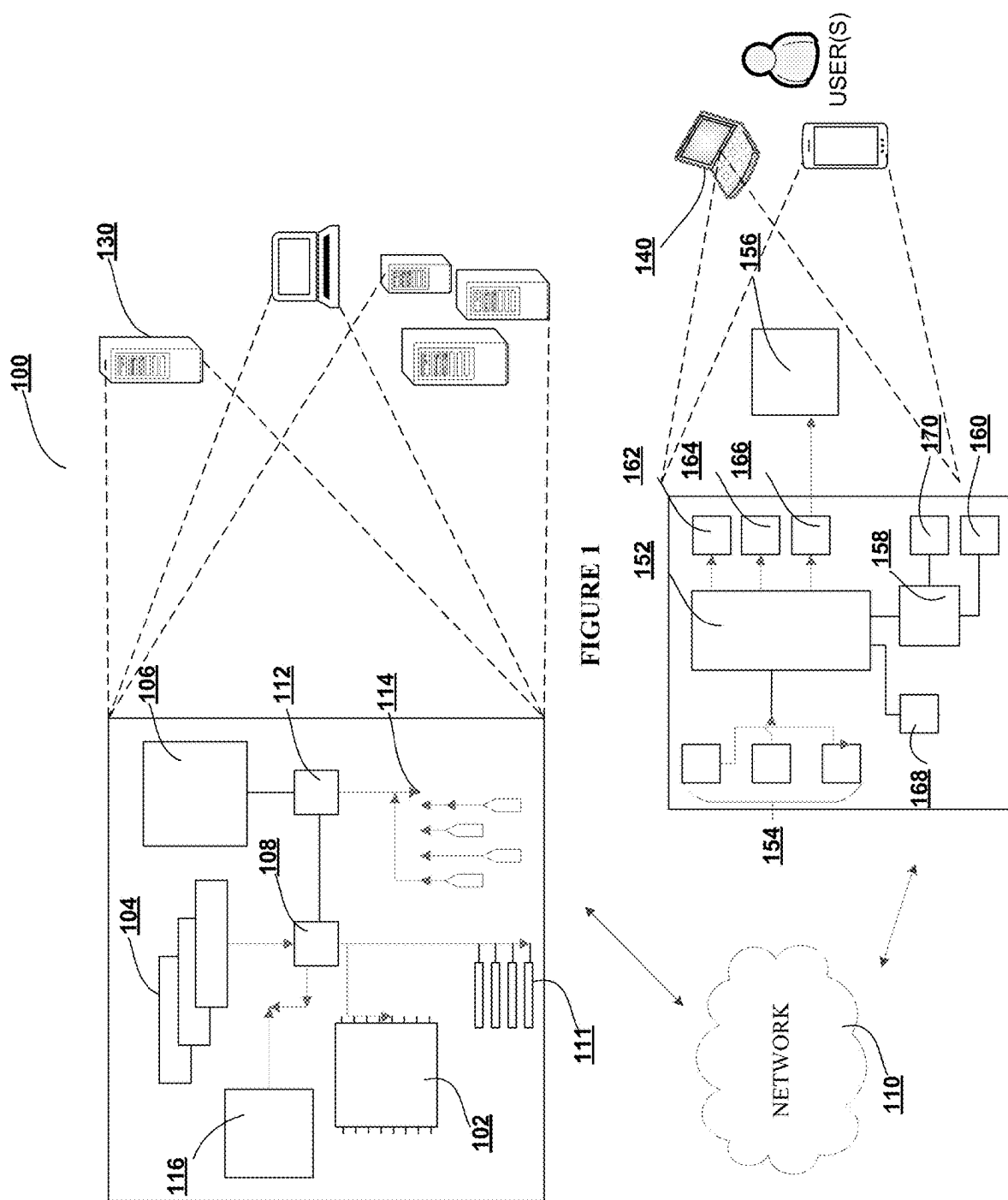
Publication Classification

(51) **Int. Cl.**
G06F 8/658 (2018.01)

(57) **ABSTRACT**

Embodiments of the invention are directed to systems, methods, and computer program products for automatically enabling features in a software production environment. In some embodiments, the method includes defining, using a configuration scanning function, a plurality of features in a production environment, where each feature of the plurality of features includes a feature activation status, and activating, during a code delivery process, at least one feature of the plurality of features based on an output of a machine learning algorithm. The method may also include defining, using a code scanning function, a set of system requirements associated with a source code. The configuration scanning function and the code scanning functions may each be configured to provide at least one input of the machine learning algorithm.





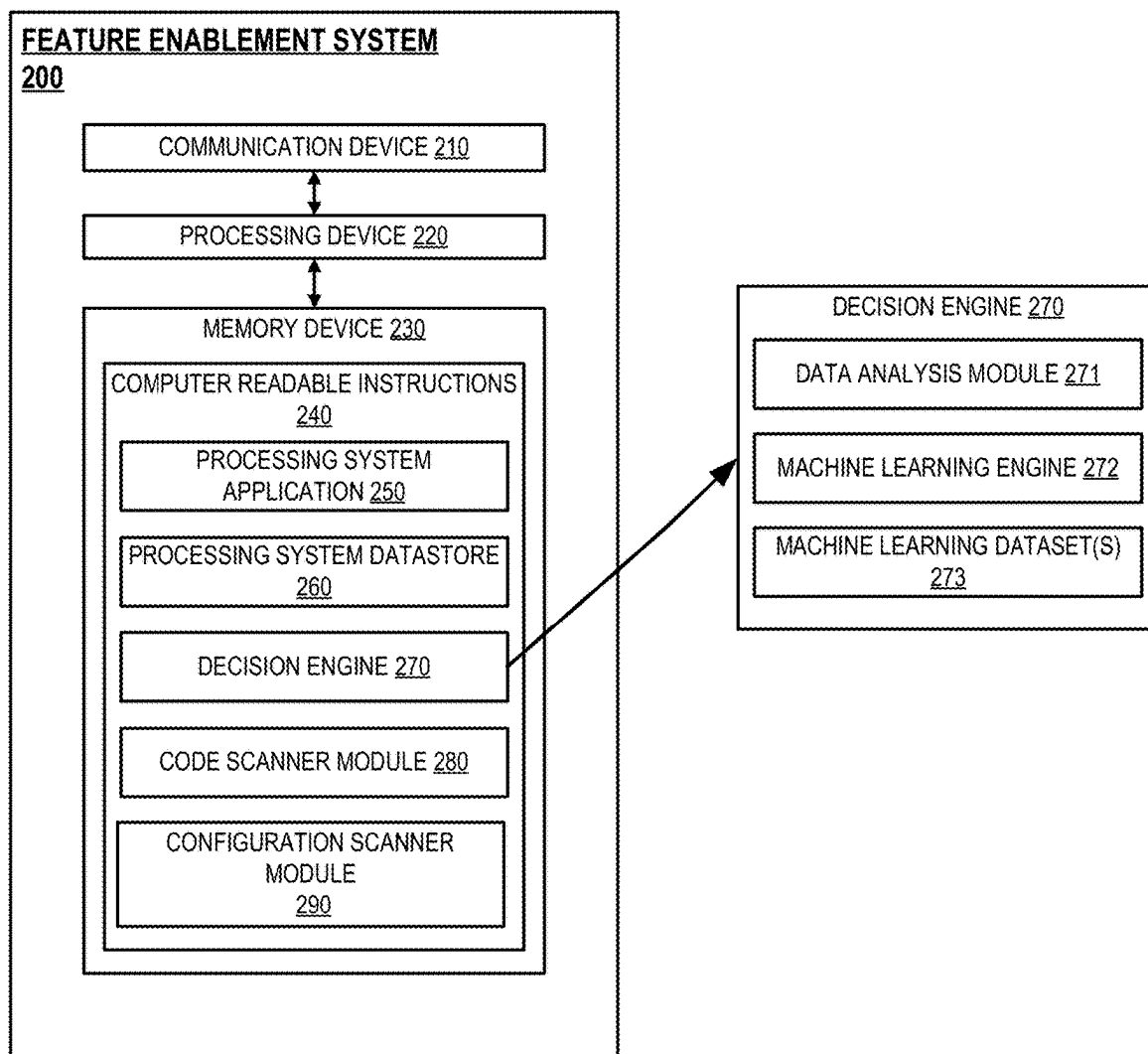
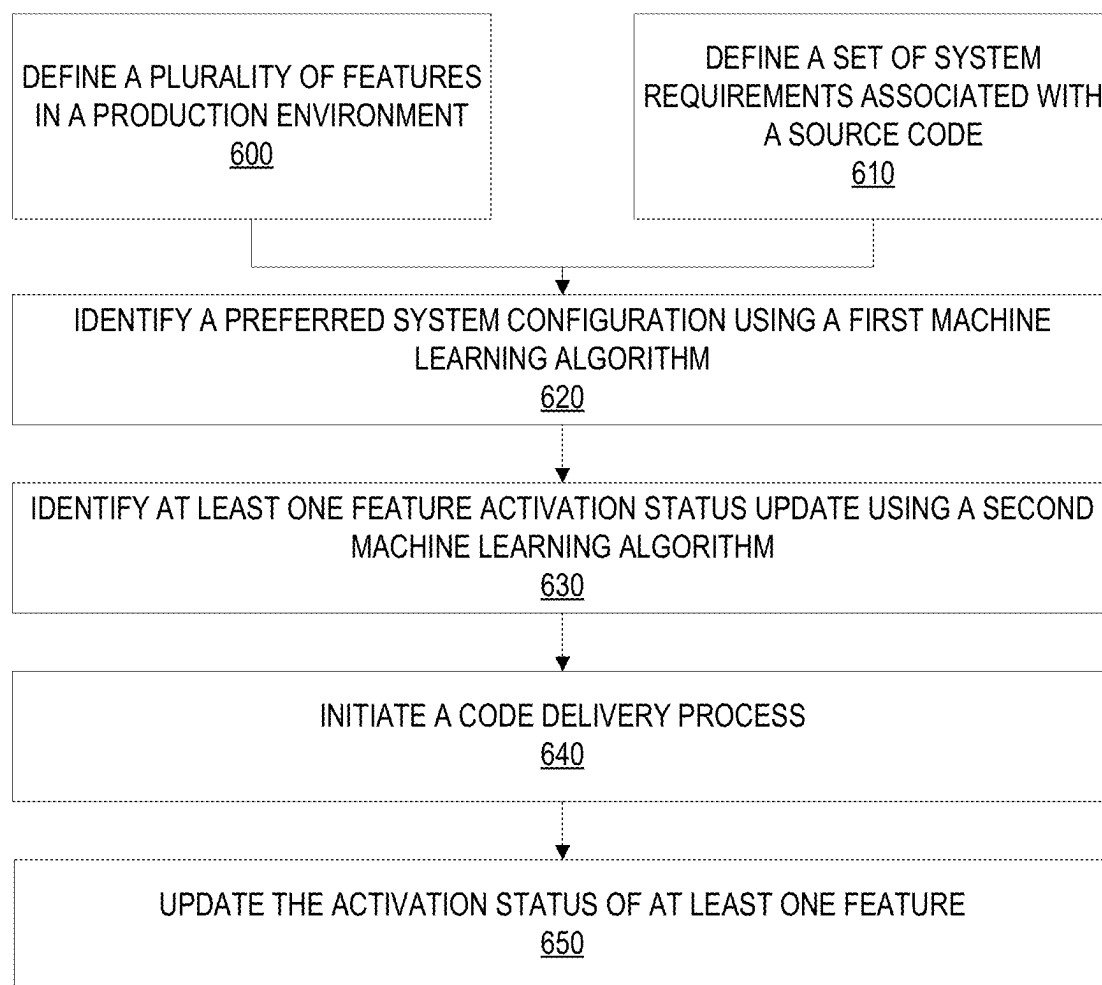


FIG. 2

**FIG. 3**

**SYSTEM AND METHOD FOR
AUTOMATICALLY ENABLING FEATURES
IN A SOFTWARE PRODUCTION
ENVIRONMENT**

FIELD OF THE INVENTION

[0001] The present invention embraces a system for automatically enabling features in a software production environment.

BACKGROUND

[0002] There is a need for a system that automatically resolves feature discrepancies discovered during integration of software packages into production environments.

BRIEF SUMMARY

[0003] The following presents a simplified summary of one or more embodiments of the invention in order to provide a basic understanding of such embodiments. This summary is not an extensive overview of all contemplated embodiments, and is intended to neither identify key or critical elements of all embodiments, nor delineate the scope of any or all embodiments. Its sole purpose is to present some concepts of one or more embodiments in a simplified form as a prelude to the more detailed description that is presented later.

[0004] Embodiments of the invention relate to systems, methods, and computer program products for automatically enabling features in a production environment, the invention including: defining, using a configuration scanning function, a plurality of features in a production environment, where each feature of the plurality of features includes a feature activation status; and activating, during a code delivery process, at least one feature of the plurality of features based on an output of a machine learning algorithm.

[0005] In some embodiments, the invention further includes defining, using a code scanning function, a set of system requirements associated with a source code.

[0006] In some embodiments, the code delivery process comprises integrating the source code into the production environment.

[0007] In some embodiments, the configuration scanning function is configured to provide at least one input of the machine learning algorithm.

[0008] In some embodiments, the code scanning function is configured to provide at least one input of the machine learning algorithm.

[0009] In some embodiments, the machine learning algorithm is configured to define a preferred system configuration.

[0010] In some embodiments, the machine learning algorithm is further configured to define at least one feature activation status associated with the preferred system configuration.

[0011] In some embodiments, activating the at least one feature of the plurality of features includes updating the feature activation status of the at least one feature.

[0012] The features, functions, and advantages that have been discussed may be achieved independently in various embodiments of the present invention or may be combined with yet other embodiments, further details of which can be seen with reference to the following description and drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0013] Having thus described embodiments of the invention in general terms, reference will now be made to the accompanying drawings, wherein:

[0014] FIG. 1 illustrates technical components of a system for automatically enabling features in a production environment, in accordance with one embodiment of the present disclosure;

[0015] FIG. 2 is a block diagram illustrating the system for automatically enabling features; and

[0016] FIG. 3 illustrates a process flow for automatically enabling features in a production environment, in accordance with one embodiment of the present disclosure.

**DETAILED DESCRIPTION OF EMBODIMENTS
OF THE INVENTION**

[0017] Embodiments of the present invention will now be described more fully hereinafter with reference to the accompanying drawings, in which some, but not all, embodiments of the invention are shown. Indeed, the invention may be embodied in many different forms and should not be construed as limited to the embodiments set forth herein; rather, these embodiments are provided so that this disclosure will satisfy applicable legal requirements. Like numbers refer to elements throughout. Where possible, any terms expressed in the singular form herein are meant to also include the plural form and vice versa, unless explicitly stated otherwise. Also, as used herein, the term “a” and/or “an” shall mean “one or more,” even though the phrase “one or more” is also used herein.

[0018] As used herein, an “entity” may be any institution employing information technology resources and particularly technology infrastructure configured for managing electronic workflows. Typically, these workflows can be related to the people who work for the organization, its products or services, the customers or any other aspect of the operations of the organization. As such, the entity may be any institution, group, association, financial institution, establishment, company, union, authority or the like, employing information technology resources for managing electronic workflows.

[0019] As described herein, a “user” may be an individual associated with an entity. As such, in some embodiments, the user may be an individual having past relationships, current relationships or potential future relationships with an entity. In some embodiments, a “user” may be an employee (e.g., an associate, a project manager, an IT specialist, a manager, an administrator, an internal operations analyst, or the like) of the entity or enterprises affiliated with the entity, capable of operating the systems described herein. In some embodiments, a “user” may be any individual, entity or system who has a relationship with the entity, such as a customer or a prospective customer. In other embodiments, a user may be a system performing one or more tasks described herein.

[0020] As used herein, a “user interface” may be any device or software that allows a user to input information, such as commands or data, into a device, or that allows the device to output information to the user. For example, the user interface includes a graphical user interface (GUI) or an interface to input computer-executable instructions that direct a processing device to carry out specific functions. The user interface typically employs certain input and output devices to input data received from a user second user or

output data to a user. These input and output devices may include a display, mouse, keyboard, button, touchpad, touch screen, microphone, speaker, LED, light, joystick, switch, buzzer, bell, and/or other user input/output device for communicating with one or more users.

[0021] As used herein, an “engine” may refer to core elements of a computer program, or part of a computer program that serves as a foundation for a larger piece of software and drives the functionality of the software. An engine may be self-contained, but externally-controllable code that encapsulates powerful logic designed to perform or execute a specific type of function. In one aspect, an engine may be underlying source code that establishes file hierarchy, input and output methods, and how a specific part of a computer program interacts or communicates with other software and/or hardware. The specific components of an engine may vary based on the needs of the specific computer program as part of the larger piece of software. In some embodiments, an engine may be configured to retrieve resources created in other computer programs, which may then be ported into the engine for use during specific operational aspects of the engine. An engine may be configurable to be implemented within any general purpose computing system. In doing so, the engine may be configured to execute source code embedded therein to control specific features of the general purpose computing system to execute specific computing operations, thereby transforming the general purpose system into a specific purpose computing system.

[0022] It should also be understood that “operatively coupled,” as used herein, means that the components may be formed integrally with each other, or may be formed separately and coupled together. Furthermore, “operatively coupled” means that the components may be formed directly to each other, or to each other with one or more components located between the components that are operatively coupled together. Furthermore, “operatively coupled” may mean that the components are detachable from each other, or that they are permanently coupled together. Furthermore, operatively coupled components may mean that the components retain at least some freedom of movement in one or more directions or may be rotated about an axis (i.e., rotationally coupled, pivotally coupled). Furthermore, “operatively coupled” may mean that components may be electronically connected and/or in fluid communication with one another.

[0023] As used herein, an “interaction” may refer to any communication between one or more users, one or more entities or institutions, and/or one or more devices, nodes, clusters, or systems within the system environment described herein. For example, an interaction may refer to a transfer of data between devices, an accessing of stored data by one or more nodes of a computing cluster, a transmission of a requested task, or the like.

[0024] As used herein, “production environment” includes various components used to deploy, implement, access, and use, a given application as that application is intended to be used. In various embodiments, production environments include multiple production environment components that are combined; communicatively coupled; virtually and/or physically connected; and/or associated with one another, to provide the production environment implementing the application. In some embodiments, the production environment components making up a given production environment can

include, but are not limited to, one or more computing environments used to implement the application in the production environment such as a data center, a cloud computing environment, and/or one or more other computing environments in which one or more components and/or services used by the application in the production environment are implemented; one or more computing systems or computing entities used to implement the application in the production environment; one or more supervisory or control systems, such as hypervisors, used to implement the application in the production environment; one or more communications channels used to implement the application in the production environment; one or more access control systems, such as firewalls and gateways, used to implement the application in the production environment; one or more routing systems, such as routers and switches, used to implement the application in the production environment; one or more communications endpoint proxy systems, such as load balancers or buffers, used to implement the application in the production environment; one or more traffic or access control systems used to implement the application in the production environment; one or more secure communication protocols and/or endpoints, such as Secure Sockets Layer (SSL) protocols, used to implement the application in the production environment; one or more databases used to implement the application in the production environment; one or more internal or external services used to implement the application in the production environment; one or more backend systems, such as backend servers or other hardware used to implement the application in the production environment; one or more software systems used to implement the application in the production environment; and/or any other components making up an actual production environment in which an application is to be deployed, implemented, accessed, and run, as discussed herein, and/or as known in the art at the time of filing, and/or as developed after the time of filing.

[0025] As used herein, “machine learning algorithms” may refer to programs (math and logic) that are configured to self-adjust and perform better as they are exposed to more data. To this extent, machine learning algorithms are capable of adjusting their own parameters, given feedback on previous performance in making a prediction about a dataset. Machine learning algorithms contemplated, described, and/or used herein include supervised learning (e.g., using logistic regression, using back propagation neural networks, using random forests, decision trees, and the like), unsupervised learning (e.g., using an Apriori algorithm, using K-means clustering), semi-supervised learning, reinforcement learning (e.g., using a Q-learning algorithm, using temporal difference learning), and/or any other suitable machine learning model types. Each of these types of machine learning algorithms can implement any of one or more of a regression algorithm (e.g., ordinary least squares, logistic regression, stepwise regression, multivariate adaptive regression splines, locally estimated scatterplot smoothing, and the like), an instance-based method (e.g., k-nearest neighbor, learning vector quantization, self-organizing map, and the like), a regularization method (e.g., ridge regression, least absolute shrinkage and selection operator, elastic net, and the like), a decision tree learning method (e.g., classification and regression tree, C4.5, chi-squared automatic interaction detection, decision stump, random forest, multivariate adaptive regression splines, gradient boosting

machines, and the like), a Bayesian method (e.g., naïve Bayes, averaged one-dependence estimators, Bayesian belief network, and the like), a kernel method (e.g., a support vector machine, a radial basis function, a linear analysis, and the like), a clustering method (e.g., k-means clustering, expectation maximization, and the like), an associated rule learning algorithm, an artificial neural network model (e.g., a Perceptron method, a back-propagation method, a Hopfield network method, a self-organizing map method, a learning vector quantization method, and the like), a deep learning algorithm (e.g., a deep belief network method, a convolution network method, a stacked auto-encoder method, and the like), a dimensionality reduction method (e.g., principal component analysis, partial least squares regression, multidimensional scaling, projection pursuit, and the like), an ensemble method (e.g., boosting, bootstrapped aggregation, stacked generalization, gradient boosting machine method, random forest method, and the like), and/or any suitable form of machine learning algorithm.

[0026] As used herein, “machine learning model” may refer to a mathematical model generated by machine learning algorithms based on sample data, known as training data, to make predictions or decisions without being explicitly programmed to do so. The machine learning model represents what was learned by the machine learning algorithm and represents the rules, numbers, and any other algorithm-specific data structures required to for classification.

[0027] In conventional systems, issues arise for independent software vendors (ISVs) when their software is being manipulated and changed by recipient systems for the software packets to be integrated and implemented into the recipient system’s production environment. Thus, the present invention provides a solution to improve recipient system requirements or recipient system onboarding processes from the ISV perspective. Specifically, the invention comprises a series of machine learning algorithms that analyze data from the production environment in the recipient’s system and the data from the ISV regarding the software solution (e.g., the name of the solution, the data that was certified for the software solution, the features of the software solution, the functionalities of the software solution, and/or the like), in order to identify fields within the onboarding process need to be enabled. The invention is further configured to automatically enable said fields.

[0028] FIG. 1 presents an exemplary block diagram of the system environment for for automatically enabling features in a production environment 100, in accordance with an embodiment of the invention. FIG. 1 provides a unique system that includes specialized servers and system communicably linked across a distributive network of nodes required to perform the functions of the process flows described herein in accordance with embodiments of the present invention.

[0029] As illustrated, the system environment 100 includes a network 110, a system 130, and a user input system 140. Also shown in FIG. 1 is one or more user(s) of the user input system 140. The user input system 140 is intended to represent various forms of mobile devices, such as laptops, personal digital assistants, augmented reality (AR) devices, virtual reality (VR) devices, extended reality (XR) devices, and/or the like, and non-mobile devices such as desktops, video recorders, audio/video player, radio, workstations, and/or the like. The user may be a person who uses the user input system 140 to execute one or more

processes described herein using one or more applications stored thereon. The one or more applications may be configured to communicate with the system 130, execute a process or method, input information onto a user interface presented on the user input system 140, or the like. The applications stored on the user input system 140 and the system 130 may incorporate one or more parts of any process flow described herein.

[0030] As shown in FIG. 1, the system 130, and the user input system 140 are each operatively and selectively connected to the network 110, which may include one or more separate networks. In addition, the network 110 may include a telecommunication network, local area network (LAN), a wide area network (WAN), and/or a global area network (GAN), such as the Internet. It will also be understood that the network 110 may be secure and/or unsecure and may also include wireless and/or wired and/or optical interconnection technology.

[0031] In some embodiments, the system 130 and the user input system 140 may be used to implement the processes described herein, including the mobile-side and server-side processes for installing a computer program from a mobile device to a computer, in accordance with an embodiment of the present invention. The system 130 is intended to represent various forms of digital computers, such as laptops, desktops, workstations, electronic kiosk devices, blade servers, mainframes, or any combination of the aforementioned. The user input system 140 is intended to represent various forms of personal devices, such as laptops, desktops, mobile devices, smartphones, and other similar computing devices. The components shown here, their connections and relationships, and their functions, are meant to be exemplary only, and are not meant to limit implementations of the inventions described and/or claimed in this document.

[0032] In accordance with some embodiments, the system 130 may include a processor 102, memory 104, a storage device 106, a high-speed interface 108 connecting to memory 104, and a low-speed interface 112 connecting to low speed bus 114 and storage device 106. Each of the components 102, 104, 106, 108, 111, and 112 are interconnected using various buses, and may be mounted on a common motherboard or in other manners as appropriate. The processor 102 can process instructions for execution within the system 130, including instructions stored in the memory 104 or on the storage device 106 to display graphical information for a GUI on an external input/output device, such as display 116 coupled to a high-speed interface 108. In other implementations, multiple processors and/or multiple buses may be used, as appropriate, along with multiple memories and types of memory. Also, multiple systems, same or similar to system 130 may be connected, with each system providing portions of the necessary operations (e.g., as a server bank, a group of blade servers, or a multi-processor system). In some embodiments, the system 130 may be a server managed by an entity. The system 130 may be located at a facility associated with the entity or remotely from the facility associated with the entity.

[0033] The memory 104 stores information within the system 130. In one implementation, the memory 104 is a volatile memory unit or units, such as volatile random access memory (RAM) having a cache area for the temporary storage of information. In another implementation, the memory 104 is a non-volatile memory unit or units. The memory 104 may also be another form of computer-readable

medium, such as a magnetic or optical disk, which may be embedded and/or may be removable. The non-volatile memory may additionally or alternatively include an EEPROM, flash memory, and/or the like. The memory 104 may store any one or more of pieces of information and data used by the system in which it resides to implement the functions of that system. In this regard, the system may dynamically utilize the volatile memory over the non-volatile memory by storing multiple pieces of information in the volatile memory, thereby reducing the load on the system and increasing the processing speed.

[0034] The storage device 106 is capable of providing mass storage for the system 130. In one aspect, the storage device 106 may be or contain a computer-readable medium, such as a floppy disk device, a hard disk device, an optical disk device, or a tape device, a flash memory or other similar solid state memory device, or an array of devices, including devices in a storage area network or other configurations. A computer program product can be tangibly embodied in an information carrier. The computer program product may also contain instructions that, when executed, perform one or more methods, such as those described above. The information carrier may be a non-transitory computer- or machine-readable storage medium, such as the memory 104, the storage device 104, or memory on processor 102.

[0035] In some embodiments, the system 130 may be configured to access, via the 110, a number of other computing devices (not shown). In this regard, the system 130 may be configured to access one or more storage devices and/or one or more memory devices associated with each of the other computing devices. In this way, the system 130 may implement dynamic allocation and de-allocation of local memory resources among multiple computing devices in a parallel or distributed system. Given a group of computing devices and a collection of interconnected local memory devices, the fragmentation of memory resources is rendered irrelevant by configuring the system 130 to dynamically allocate memory based on availability of memory either locally, or in any of the other computing devices accessible via the network. In effect, it appears as though the memory is being allocated from a central pool of memory, even though the space is distributed throughout the system. This method of dynamically allocating memory provides increased flexibility when the data size changes during the lifetime of an application and allows memory reuse for better utilization of the memory resources when the data sizes are large.

[0036] The high-speed interface 108 manages bandwidth-intensive operations for the system 130, while the low speed controller 112 manages lower bandwidth-intensive operations. Such allocation of functions is exemplary only. In some embodiments, the high-speed interface 108 is coupled to memory 104, display 116 (e.g., through a graphics processor or accelerator), and to high-speed expansion ports 111, which may accept various expansion cards (not shown). In such an implementation, low-speed controller 112 is coupled to storage device 106 and low-speed expansion port 114. The low-speed expansion port 114, which may include various communication ports (e.g., USB, Bluetooth, Ethernet, wireless Ethernet), may be coupled to one or more input/output devices, such as a keyboard, a pointing device, a scanner, or a networking device such as a switch or router, e.g., through a network adapter.

[0037] The system 130 may be implemented in a number of different forms, as shown in FIG. 1. For example, it may be implemented as a standard server, or multiple times in a group of such servers. Additionally, the system 130 may also be implemented as part of a rack server system or a personal computer such as a laptop computer. Alternatively, components from system 130 may be combined with one or more other same or similar systems and an entire system 140 may be made up of multiple computing devices communicating with each other.

[0038] FIG. 1 also illustrates a user input system 140, in accordance with an embodiment of the invention. The user input system 140 includes a processor 152, memory 154, an input/output device such as a display 156, a communication interface 158, and a transceiver 160, among other components. The user input system 140 may also be provided with a storage device, such as a microdrive or other device, to provide additional storage. Each of the components 152, 154, 158, and 160, are interconnected using various buses, and several of the components may be mounted on a common motherboard or in other manners as appropriate.

[0039] The processor 152 is configured to execute instructions within the user input system 140, including instructions stored in the memory 154. The processor may be implemented as a chipset of chips that include separate and multiple analog and digital processors. The processor may be configured to provide, for example, for coordination of the other components of the user input system 140, such as control of user interfaces, applications run by user input system 140, and wireless communication by user input system 140.

[0040] The processor 152 may be configured to communicate with the user through control interface 164 and display interface 166 coupled to a display 156. The display 156 may be, for example, a TFT LCD (Thin-Film-Transistor Liquid Crystal Display) or an OLED (Organic Light Emitting Diode) display, or other appropriate display technology. The display interface 156 may comprise appropriate circuitry and configured for driving the display 156 to present graphical and other information to a user. The control interface 164 may receive commands from a user and convert them for submission to the processor 152. In addition, an external interface 168 may be provided in communication with processor 152, so as to enable near area communication of user input system 140 with other devices. External interface 168 may provide, for example, for wired communication in some implementations, or for wireless communication in other implementations, and multiple interfaces may also be used.

[0041] The memory 154 stores information within the user input system 140. The memory 154 can be implemented as one or more of a computer-readable medium or media, a volatile memory unit or units, or a non-volatile memory unit or units. Expansion memory may also be provided and connected to user input system 140 through an expansion interface (not shown), which may include, for example, a SIMM (Single In Line Memory Module) card interface. Such expansion memory may provide extra storage space for user input system 140, or may also store applications or other information therein. In some embodiments, expansion memory may include instructions to carry out or supplement the processes described above, and may include secure information also. For example, expansion memory may be provided as a security module for user input system 140, and

may be programmed with instructions that permit secure use of user input system **140**. In addition, secure applications may be provided via the SIMM cards, along with additional information, such as placing identifying information on the SIMM card in a non-hackable manner. In some embodiments, the user may use the applications to execute processes described with respect to the process flows described herein. Specifically, the application executes the process flow discussed in greater detail with respect to FIG. 2. It will be understood that the one or more applications stored in the system **130** and/or the user computing system **140** may interact with one another and may be configured to implement any one or more portions of the various user interfaces and/or process flow described herein.

[0042] The memory **154** may include, for example, flash memory and/or NVRAM memory. In one aspect, a computer program product is tangibly embodied in an information carrier. The computer program product contains instructions that, when executed, perform one or more methods, such as those described herein. The information carrier is a computer- or machine-readable medium, such as the memory **154**, expansion memory, memory on processor **152**, or a propagated signal that may be received, for example, over transceiver **160** or external interface **168**.

[0043] In some embodiments, the user may use the user input system **140** to transmit and/or receive information or commands to and from the system **130**. In this regard, the system **130** may be configured to establish a communication link with the user input system **140**, whereby the communication link establishes a data channel (wired or wireless) to facilitate the transfer of data between the user input system **140** and the system **130**. In doing so, the system **130** may be configured to access one or more aspects of the user input system **140**, such as, a GPS device, an image capturing component (e.g., camera), a microphone, a speaker, or the like.

[0044] The user input system **140** may communicate with the system **130** (and one or more other devices) wirelessly through communication interface **158**, which may include digital signal processing circuitry where necessary. Communication interface **158** may provide for communications under various modes or protocols, such as GSM voice calls, SMS, EMS, or MMS messaging, CDMA, TDMA, PDC, WCDMA, CDMA2000, or GPRS, among others. Such communication may occur, for example, through radio-frequency transceiver **160**. In addition, short-range communication may occur, such as using a Bluetooth, Wi-Fi, or other such transceiver (not shown). In addition, GPS (Global Positioning System) receiver module **170** may provide additional navigation- and location-related wireless data to user input system **140**, which may be used as appropriate by applications running thereon, and in some embodiments, one or more applications operating on the system **130**.

[0045] The user input system **140** may also communicate audibly using audio codec **162**, which may receive spoken information from a user and convert it to usable digital information. Audio codec **162** may likewise generate audible sound for a user, such as through a speaker, e.g., in a handset of user input system **140**. Such sound may include sound from voice telephone calls, may include recorded sound (e.g., voice messages, music files, etc.) and may also include sound generated by one or more applications oper-

ating on the user input system **140**, and in some embodiments, one or more applications operating on the system **130**.

[0046] Various implementations of the systems and techniques described here can be realized in digital electronic circuitry, integrated circuitry, specially designed ASICs (application specific integrated circuits), computer hardware, firmware, software, and/or combinations thereof. These various implementations can include implementation in one or more computer programs that are executable and/or interpretable on a programmable system including at least one programmable processor, which may be special or general purpose, coupled to receive data and instructions from, and to transmit data and instructions to, a storage system, at least one input device, and at least one output device.

[0047] These computer programs (also known as programs, software, software applications or code) include machine instructions for a programmable processor and can be implemented in a high-level procedural and/or object-oriented programming language, and/or in assembly/machine language. As used herein, the terms “machine-readable medium” “computer-readable medium” refers to any computer program product, apparatus and/or device (e.g., magnetic discs, optical disks, memory, Programmable Logic Devices (PLDs)) used to provide machine instructions and/or data to a programmable processor, including a machine-readable medium that receives machine instructions as a machine-readable signal. The term “machine-readable signal” refers to any signal used to provide machine instructions and/or data to a programmable processor.

[0048] To provide for interaction with a user, the systems and techniques described here can be implemented on a computer having a display device (e.g., a CRT (cathode ray tube) or LCD (liquid crystal display) monitor) for displaying information to the user and a keyboard and a pointing device (e.g., a mouse or a trackball) by which the user can provide input to the computer. Other kinds of devices can be used to provide for interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback (e.g., visual feedback, auditory feedback, or tactile feedback); and input from the user can be received in any form, including acoustic, speech, or tactile input.

[0049] The systems and techniques described here can be implemented in a computing system that includes a back end component (e.g., as a data server), or that includes a middleware component (e.g., an application server), or that includes a front end component (e.g., a client computer having a graphical user interface or a Web browser through which a user can interact with an implementation of the systems and techniques described here), or any combination of such back end, middleware, or front end components. The components of the system can be interconnected by any form or medium of digital data communication (e.g., a communication network). Examples of communication networks include a local area network (“LAN”), a wide area network (“WAN”), and the Internet.

[0050] The computing system can include clients and servers. A client and server are generally remote from each other and typically interact through a communication network. The relationship of client and server arises by virtue of computer programs running on the respective computers and having a client-server relationship to each other.

[0051] It will be understood that the embodiment of the system environment illustrated in FIG. 1 is exemplary and that other embodiments may vary. As another example, in some embodiments, the system 130 includes more, less, or different components. As another example, in some embodiments, some or all of the portions of the system environment 100 may be combined into a single portion. Likewise, in some embodiments, some or all of the portions of the system 130 may be separated into two or more distinct portions.

[0052] FIG. 2 illustrates a block diagram of a feature enablement system 200 associated with the system environment 100, in accordance with embodiments of the present invention. As illustrated in FIG. 2, the feature enablement system 200 may include a communication device 210, a processing device 220, and a memory device 230 having a decision engine 270, a code scanner module 280, a configuration scanner module 290, a processing system application 250 and a processing system datastore 260 stored therein. As shown, the processing device 220 is operatively connected to and is configured to control and cause the communication device 210 and the memory device 230 to perform one or more functions. In some embodiments, the code scanner module 280, the configuration scanner module 290, the decision engine 270 and/or the processing system application 250 comprise computer readable instructions 240 that when executed by the processing device 220 cause the processing device 220 to perform one or more functions and/or transmit control instructions to other systems, applications, and/or devices in the system environment 100. It will be understood that the code scanner module 280, the configuration scanner module 290, the decision engine 270 and/or the processing system application 250 may be executable to initiate, perform, complete, and/or facilitate one or more portions of any embodiments described and/or contemplated herein.

[0053] The decision engine 270 may further comprise a data analysis module 271, a machine learning engine 272, and a machine learning dataset(s) 273. The data analysis module 271 may store instructions and/or data that may cause or enable the feature enablement system 200 to receive, store, and/or analyze data received code scanner module 280 and/or configuration scanner module 290. The data analysis module may process data and/or metadata to identify preferred configurations as will be further discussed in FIG. 3. The machine learning engine 272 and machine learning dataset(s) 273 may store instructions and/or data that cause or enable the feature enablement system 200 to determine, in real-time and based on received information, one or more desired feature activation status updates. The machine learning dataset(s) 273 may contain data queried from one or more remote servers and/or may be based on historical data relating to a particular data category, data type, or the like. In some embodiments, the machine learning dataset(s) 273 may also contain data relating to a plurality of machine learning algorithms.

[0054] The machine learning engine 272 may receive data from a plurality of sources and, using one or more machine learning algorithms, may generate one or more machine learning datasets 273. Various machine learning algorithms may be used without departing from the invention as is described in greater detail herein.

[0055] The machine learning datasets 273 may include machine learning data linking one or more outputs of the code scanner module 280 with one or more outputs of the

configuration scanner module 290 to identify one or features of a production environment to be activated during a code delivery process. For example, the machine learning datasets 273 may include information linking code affecting user data (e.g. personally identifiable information and/or the like) with a one or more security features. In another example, the machine learning datasets 273 may include information linking a code module to a particular storage location (e.g. a private server rather than a cloud or edge server). The combination of this information may cause the decision engine 270 to determine that particular features of the production environment must be activated or deactivated based on the requirements of the code involved in the code delivery process.

[0056] The communication device 210 may generally include a modem, server, transceiver, and/or other devices for communicating with other devices on the network 101. The communication device 210 may be a communication interface having one or more communication devices configured to communicate with one or more other devices on the network 101.

[0057] Additionally, referring to the feature enablement system 200 illustrated in FIG. 2, the processing device 220 may generally refer to a device or combination of devices having circuitry used for implementing the communication and/or logic functions of the feature enablement system 200. For example, the processing device 220 may include a control unit, a digital signal processor device, a microprocessor device, and various analog-to-digital converters, digital-to-analog converters, and other support circuits and/or combinations of the foregoing. Control and signal processing functions of the data obfuscation system 200 may be allocated between these processing devices according to their respective capabilities. The processing device 220 may further include functionality to operate one or more software programs based on computer-executable program code 240 thereof, which may be stored in a memory device 230, such as the processing system application 250 and the decision engine 270. As the phrase is used herein, a processing device may be “configured to” perform a certain function in a variety of ways, including, for example, by having one or more general-purpose circuits perform the function by executing particular computer-executable program code embodied in computer-readable medium, and/or by having one or more application-specific circuits perform the function. The processing device 220 may be configured to use the network communication interface of the communication device 210 to transmit and/or receive data and/or commands to and/or from the other devices/systems connected to the network 101.

[0058] The memory device 230 within the feature enablement system 200 may generally refer to a device or combination of devices that store one or more forms of computer-readable media for storing data and/or computer-executable program code/instructions. For example, the memory device 230 may include any computer memory that provides an actual or virtual space to temporarily or permanently store data and/or commands provided to the processing device 220 when it carries out its functions described herein. As used herein, memory may include any computer readable medium configured to store data, code, or other information. The memory device 350 may include volatile memory, such as volatile Random Access Memory (RAM) including a cache area for the temporary storage of data. The

memory device **420** may also include non-volatile memory, which can be embedded and/or may be removable. The non-volatile memory may additionally or alternatively include an electrically erasable programmable read-only memory (EEPROM), flash memory or the like.

[0059] In some instances, various features and functions of the invention are described herein with respect to a “system.” In some instances, the system may refer to the feature enablement system **200** performing one or more steps described herein in conjunction with other devices and systems, either automatically based on executing computer readable instructions of the memory device **230**, or in response to receiving control instructions from another device in the system environment **100**. In some instances, the system refers to the devices and systems on the system environment **100** of FIG. **1**. The features and functions of various embodiments of the invention are described below in further detail. It is understood that the servers, systems, and devices described herein illustrate one embodiment of the invention. It is further understood that one or more of the servers, systems, and devices can be combined in other embodiments and still function in the same or similar way as the embodiments described herein.

[0060] FIG. **3** is a high-level process flow diagram illustrating a process using the feature enablement system, in accordance with one embodiment of the present disclosure. The process begins at block **600**, where the configuration scanner module **290** defines a plurality of features in a production environment. The configuration scanner module **290** may be configured to read data and metadata (e.g., source code) associated with a production environment to identify and define the plurality of features. Each feature may comprise any functionality of the production environment, such as a security setting, access to internal or external data, configuration setting, application status, user interface functionality, and/or the like. In some embodiments, the definition of each feature comprises a feature activation status. For example, the configuration scanner module **290** may be configured to read data associated with a production environment and define a first feature, the first feature being associated with user access to data from a particular cloud server. If the current production environment does not allow user access to data from the particular cloud server, the configuration scanner module **290** may define the first feature as inactive. In some embodiments, the feature activation status comprises one or more rules (i.e. logical statements) associated with the feature, where the feature activation status is defined as active when the rule evaluates as true and the feature activation status is defined as inactive when the rule evaluates as false.

[0061] The process may then continue to block **610**, where the code scanner module **280** defines a set of system requirements associated with a source code. In some embodiments, depending on the available computational power of the system **200**, blocks **600** and **610** occur simultaneously. The code scanner module **280** may be configured to read source code associated with a change to the production environment (e.g., a new application integration, application update, security protocol update, and/or the like). The code scanner module **280** may then identify and define the set of system requirements associated with the source code. Each system requirement may comprise any functionality of the production environment, such as a security setting, access to internal or external data, configuration setting,

application status, user interface functionality, and/or the like. For example, the code scanner module **280** may be configured to read source code associated with a new application integration and define a first system requirement, the first system requirement being associated with system access to data from a particular cloud server.

[0062] The process may then continue to block **620**, where the decision engine **270** may identify and define a preferred system configuration using a first machine learning algorithm. As described with respect to FIG. **2**, the decisions engine **270** may first execute the data analysis module **271**, which may access data associated with the plurality of features defined by the configuration scanner module **290**. The data analysis module **271** may further access data associated with the set of system requirements defined by the code scanner module **290**. The data obtained by the data analysis module **271** from the configuration scanner module **290** and the code scanner module **280** may then serve as inputs to the machine learning engine **272**. The machine learning engine **272** may then apply a first machine learning algorithm to a combination of data from the data analysis module **271** to generate one or more first machine learning datasets **273**. The first machine learning dataset **273** may define a preferred system configuration of the production environment, where the preferred system configuration defines a required feature activation status for each of the plurality of features, based on the defined set of system requirements. For example, a first system requirement may require access to data from a particular cloud server. Thus, the first machine learning dataset **273** may identify a feature associated with access to said cloud server, and may define the required feature activation status as “active.”

[0063] The process may then continue to block **630**, where the decision engine **270** may identify at least one feature activation status update using a second machine learning algorithm. In some embodiments, the machine learning engine **272** may apply a second machine learning algorithm a combination of data from the data analysis module **271** and the first machine learning dataset **273** to generate one or more second machine learning datasets **273**. The second machine learning dataset **273** may identify and define one or more features where a current feature activation status does not match a required feature activation status. Thus, the second machine learning dataset may define features with a current activation status which is incompatible with the preferred system configuration.

[0064] The process may then continue to block **640**, where the system is configured to initiate a code delivery process to integrate the source code into the production environment. In some embodiments, the code delivery process comprises a continuous integration and continuous delivery (“CI/CD”) process as is known in the art. Additionally or alternatively, the code delivery process may comprise a standalone code integration process. During the code delivery process, the process may continue to block **650**, where the system updates the feature activation status of at least one feature based on the second machine learning dataset. For example, continuing with the example of feature relating to cloud server access, the system may be configured to automatically change the feature activation status of said feature from “inactive” to “active” during the code delivery process. Thus, the system may set the production environment to match the preferred system configuration at the same time that the source code is being integrated into the

production environment, reducing the likelihood of a code error in the production environment.

[0065] As will be appreciated by one of ordinary skill in the art, the present invention may be embodied as an apparatus (including, for example, a system, a machine, a device, a computer program product, and/or the like), as a method (including, for example, a business process, a computer-implemented process, and/or the like), or as any combination of the foregoing. Accordingly, embodiments of the present invention may take the form of an entirely software embodiment (including firmware, resident software, micro-code, and the like), an entirely hardware embodiment, or an embodiment combining software and hardware aspects that may generally be referred to herein as a “system.” Furthermore, embodiments of the present invention may take the form of a computer program product that includes a computer-readable storage medium having computer-executable program code portions stored therein.

[0066] As the phrase is used herein, a processor may be “configured to” perform a certain function in a variety of ways, including, for example, by having one or more general-purpose circuits perform the function by executing particular computer-executable program code embodied in computer-readable medium, and/or by having one or more application-specific circuits perform the function.

[0067] It will be understood that any suitable computer-readable medium may be utilized. The computer-readable medium may include, but is not limited to, a non-transitory computer-readable medium, such as a tangible electronic, magnetic, optical, infrared, electromagnetic, and/or semiconductor system, apparatus, and/or device. For example, in some embodiments, the non-transitory computer-readable medium includes a tangible medium such as a portable computer diskette, a hard disk, a random access memory (RAM), a read-only memory (ROM), an erasable programmable read-only memory (EEPROM or Flash memory), a compact disc read-only memory (CD-ROM), and/or some other tangible optical and/or magnetic storage device. In other embodiments of the present invention, however, the computer-readable medium may be transitory, such as a propagation signal including computer-executable program code portions embodied therein.

[0068] It will also be understood that one or more computer-executable program code portions for carrying out the specialized operations of the present invention may be required on the specialized computer include object-oriented, scripted, and/or unscripted programming languages, such as, for example, Java, Perl, Smalltalk, C++, SQL, Python, Objective C, and/or the like. In some embodiments, the one or more computer-executable program code portions for carrying out operations of embodiments of the present invention are written in conventional procedural programming languages, such as the “C” programming languages and/or similar programming languages. The computer program code may alternatively or additionally be written in one or more multi-paradigm programming languages, such as, for example, F #.

[0069] Embodiments of the present invention are described above with reference to flowcharts and/or block diagrams. It will be understood that steps of the processes described herein may be performed in orders different than those illustrated in the flowcharts. In other words, the processes represented by the blocks of a flowchart may, in some embodiments, be performed in an order other than

the order illustrated, may be combined or divided, or may be performed simultaneously. It will also be understood that the blocks of the block diagrams illustrated, in some embodiments, merely conceptual delineations between systems and one or more of the systems illustrated by a block in the block diagrams may be combined or share hardware and/or software with another one or more of the systems illustrated by a block in the block diagrams. Likewise, a device, system, apparatus, and/or the like may be made up of one or more devices, systems, apparatuses, and/or the like. For example, where a processor is illustrated or described herein, the processor may be made up of a plurality of microprocessors or other processing devices which may or may not be coupled to one another. Likewise, where a memory is illustrated or described herein, the memory may be made up of a plurality of memory devices which may or may not be coupled to one another.

[0070] It will also be understood that the one or more computer-executable program code portions may be stored in a transitory or non-transitory computer-readable medium (e.g., a memory, and the like) that can direct a computer and/or other programmable data processing apparatus to function in a particular manner, such that the computer-executable program code portions stored in the computer-readable medium produce an article of manufacture, including instruction mechanisms which implement the steps and/or functions specified in the flowchart(s) and/or block diagram block(s).

[0071] The one or more computer-executable program code portions may also be loaded onto a computer and/or other programmable data processing apparatus to cause a series of operational steps to be performed on the computer and/or other programmable apparatus. In some embodiments, this produces a computer-implemented process such that the one or more computer-executable program code portions which execute on the computer and/or other programmable apparatus provide operational steps to implement the steps specified in the flowchart(s) and/or the functions specified in the block diagram block(s). Alternatively, computer-implemented steps may be combined with operator and/or human-implemented steps in order to carry out an embodiment of the present invention.

[0072] While certain exemplary embodiments have been described and shown in the accompanying drawings, it is to be understood that such embodiments are merely illustrative of, and not restrictive on, the broad invention, and that this invention not be limited to the specific constructions and arrangements shown and described, since various other changes, combinations, omissions, modifications and substitutions, in addition to those set forth in the above paragraphs, are possible. Those skilled in the art will appreciate that various adaptations and modifications of the just described embodiments can be configured without departing from the scope and spirit of the invention. Therefore, it is to be understood that, within the scope of the appended claims, the invention may be practiced other than as specifically described herein.

What is claimed is:

1. A system for automatically enabling features in a production environment, the system comprising:
 - at least one non-transitory storage device; and
 - at least one processing device coupled to the at least one non-transitory storage device, wherein the at least one processing device is configured to:

- define, using a configuration scanning function, a plurality of features in a production environment, wherein each feature of the plurality of features comprises a feature activation status; and
- activate, during a code delivery process, at least one feature of the plurality of features based on an output of a machine learning algorithm.
2. The system of claim 1, wherein the at least one processing device is further configured to define, using a code scanning function, a set of system requirements associated with a source code.
3. The system of claim 2, wherein the code delivery process comprises integrating the source code into the production environment.
4. The system of claim 1, wherein the configuration scanning function is configured to provide at least one input of the machine learning algorithm.
5. The system of claim 2, wherein the code scanning function is configured to provide at least one input of the machine learning algorithm.
6. The system of claim 1, wherein the machine learning algorithm is configured to define a preferred system configuration.
7. The system of claim 6, wherein the machine learning algorithm is further configured to define at least one feature activation status associated with the preferred system configuration.
8. The system of claim 1, wherein activating the at least one feature of the plurality of features comprises updating the feature activation status of the at least one feature.
9. A computer program product for automatically enabling features in a production environment, the computer program product comprising at least one non-transitory computer-readable medium having computer-readable program code portions embodied therein, the computer-readable program code portions comprising:
- an executable portion configured for defining, using a configuration scanning function, a plurality of features in a production environment, wherein each feature of the plurality of features comprises a feature activation status; and
 - an executable portion configured for activating, during a code delivery process, at least one feature of the plurality of features based on an output of a machine learning algorithm.
10. The computer program product of claim 9, further comprising an executable portion configured for defining, using a code scanning function, a set of system requirements associated with a source code.

11. The computer program product of claim 10, wherein the code delivery process comprises integrating the source code into the production environment.

12. The computer program product of claim 9, wherein the configuration scanning function is configured to provide at least one input of the machine learning algorithm.

13. The computer program product of claim 10, wherein the code scanning function is configured to provide at least one input of the machine learning algorithm.

14. The computer program product of claim 9, wherein the machine learning algorithm is configured to define a preferred system configuration.

15. The computer program product of claim 14, wherein the machine learning algorithm is further configured to define at least one feature activation status associated with the preferred system configuration.

16. The computer program product of claim 9, further comprising an executable portion configured for accessing the stored record of the established sequence of obfuscation algorithms and the unique identifier of the obfuscated dataset and utilize the stored record to de-obfuscate the obfuscated dataset.

17. A computer-implemented method for automatically enabling features in a production environment, the method comprising:

providing a computing system comprising a computer processing device and a non-transitory computer readable medium, wherein the computer readable medium comprises configured computer program instruction code, such that when said instruction code is operated by said computer processing device, said computer processing device performs the following operations:

- defining, using a configuration scanning function, a plurality of features in a production environment, wherein each feature of the plurality of features comprises a feature activation status; and
- activating, during a code delivery process, at least one feature of the plurality of features based on an output of a machine learning algorithm.

18. The method of claim 16, further comprising defining, using a code scanning function, a set of system requirements associated with a source code.

19. The method of claim 18, wherein the configuration scanning function and the code scanning functions are each configured to provide at least one input of the machine learning algorithm.

20. The method of claim 16, wherein the machine learning algorithm is configured to define a preferred system configuration and at least one feature activation status associated with the preferred system configuration.

* * * * *